High Performance Protein Prediction with ESMFold

Stone Su
Dept. of Computer Science
Columbia University
New York, NY
ss6422@columbia.edu

Layanne El Assaad
Dept. of Computer Science
Columbia University
New York, NY
lae2146@columbia.edu

***Abstract*—ESMFold is a protein structure prediction model developed by Meta in 2022, that achieves competitive accuracy with AlphaFold while eliminating the computational overhead of multiple sequence alignments. However, the cost of its forward pass scales quadratically with protein length and is dominated by memory-intensive pairwise representations and triangular attention mechanisms, limiting its ability to process long proteins. In this work, we present a comprehensive profiling and basic optimization of ESMFold, targeting the effect of sequence length on latency and memory efficiency.**

Through our profiling, we identified triangular attention, triangular multiplication, and structure module components as primary bottlenecks. We found that the greatest slowdown in the triangular attention blocks, where the percentage of total computation time doubled from 35% to >70% in long (>200 aa) proteins. To remedy this issue, we introduce framework-level optimizations including torch.compile in order to address this computational bottleneck. Additionally, we analyze the effectiveness of microbatching, demonstrating that selectively breaking down long sequences into manageable chunks can significantly reduce VRAM capacity requirements.

Our optimized implementation achieves an average of a 6% total speedup compared to the baseline ESMFold implementation, with specific kernel bottlenecks identified in the folding trunk code implementation.

I. Introduction

An essential and long ongoing question in computational biology is whether or not geometric protein structure can be accurately predicted from just a sequence of amino acids. If such a tool is possible, it would have massive implications for medicine as protein structure underlies its function, interaction affinities, and disease mechanisms. Drug discovery and antibody design would rapidly accelerate and can be done on a cost and time effective manner.

Recent advances in deep learning have enabled certain architectures to infer such structures from the massive wealth of sequence data that scientists have gathered over the years. In doing so, the reliance on experimentally determining structures has gone down, as such techniques are extremely costly. Consequently, the scale of structure prediction had exploded due to the shifting of constraints from laboratory work to computational power.

Within the family of modern protein folding models, ESMFold took a sharp diversion from the traditional multi-sequence-alignment based models like AlphaFold, in preference for a single sequence, transformer based language model. In doing so, they avoid the extreme computational requirements needed to utilize an MSA database, as well as the expensive pre-training evolutionary structural information that is needed to construct such as database. Instead of deriving explicit biological rules that govern protein geometry like AlphaFold, ESMFold focuses on implicitly learning them through training on the much easier to access sequence data. [1]

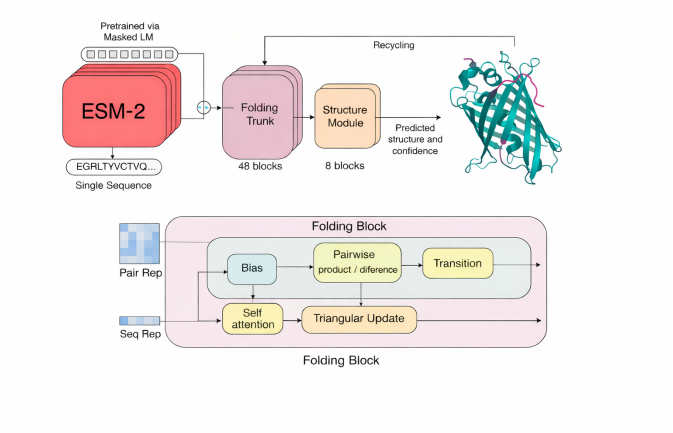

Fig. 1. Model Diagram for ESMFold [1]

ESMFold utilizes a large pretrained model to encode rich sequence embeddings (ESM2), which then get processed by a folding trunk, similar to that used by AlphaFold. This folding trunk utilizes both sequence and pairwise attention representations in order to balance between local and global structure prediction. In order to constrict its predictions with geometric consistency, ESMFold uses what is called triangular attention and multiplication, essentially utilizing residue pairs to define geometries. Finally, the structural model refines these coordinates with the learned representations from training. [1]

Despite its computational advantages over AlphaFold, ESMFold is not without its own systems-level challenges. The computational and memory costs of attention-based operations scale more than quadratically with sequence length, making it difficult to get uniform inference behavior between short (<200 residues) and long sequences (≥200 residues). For short sequences, the execution time is dominated with the constant and frequent kernel launch overheads whereas for long sequences, the execution is dominated by the triangular

attention blocks. As sequence length grows, the attention map size skyrockets, leading to intense memory requirements. This leads to the loss of predictive power in addition to certain implementations may not even be able to handle sequences of a certain length [2].

Understanding these performance characteristics is critical for both optimizing inference and enabling reliable deployment on modern GPU architectures. In this work, we conduct a detailed profiling study of ESMFold using PyTorch's profiling tools to characterize execution bottlenecks across a wide range of sequence lengths. We further investigate how compilation via torch.compile alters execution behavior, kernel fusion, and performance scaling, providing insight into how modern compiler techniques can help, or in some cases be a detriment, to the challenges posed by attention-based protein folding models.

II. Technical Approach

A. Graph Compilation with torch.compile

We apply torch.compile to ESMFold to perform graph-level optimization of the forward pass and related autograd computations. To do so, torch.compile provides a unified compilation pipeline that converts eager-mode PyTorch execution into kernels that are optimized for the specific device it is to be run on.

1) Compilation Pipeline: First, torch.compile traces the execution of Python using TorchDynamo. TorchDynamo automatically intercepts PyTorch operator calls and extracts FX graphs (i.e. intermediate representations of the code) corresponding to contiguous regions of tensor computation, while guarding on runtime properties such as tensor shapes, data types, and control-flow decisions. These graphs are converted into a lower-level representation through AOTAutograd, which maps out the forward and backward passes explicitly, enabling whole-graph optimization [3].

The lower-level graphs are then subsequently passed to TorchInductor, which performs backend-specific optimization and kernel generation. These compiled kernels are also saved so that they can be reused across compatible executions, eliminating the need to redundantly compile the same kernels over multiple inference runs. Crucially, torch.compile's pipeline is also dynamic to input shape sizes, which is invaluable to ESMFold since its inputs are protein sequence inputs of varying length [3].

2) Application to ESMFold: ESMFold's forward pass is mostly consisted of a sequence representation trunk, a pairwise representation trunk, and an iterative structure module. These components involve deeply nested tensor operations, including layer normalization, MLP blocks, triangular attention, and geometric transformations. In eager execution, these operations are executed as many individual kernels (such as add, matmul, relu), and constant switches in Python control along with excessive synchronization points. When these small kernels are constantly being launched with repetitive operations, it creates non-negligible overhead that can cause serious slowdown.

By compiling ESMFold with torch.compile, we should be able to transform large contiguous regions of sporadic computation into single fused kernels, reducing both Python overhead and kernel launch frequency. Particularly, elementwise operations and reductions within the sequence and pairwise transformer blocks are fused into single kernels. Additionally, repeated subgraphs arising from the recycling iterations in the folding trunk are optimized under a shared compiled representation [2].

For components such as triangular attention which heavily rely on attention-based kernels, these optimizations should significantly reduce launch overhead and improve memory access efficiency, which becomes a dominant factor for long sequence lengths. Additionally, TorchInductor's memory planning reduces peak activation memory by reusing buffers across fused operations [3]. This is particularly relevant for ESMFold, where quadratic and cubic scaling components (e.g., pairwise representations) impose significant restrictions when processing long proteins.

Because ESMFold was released prior to the introduction of torch.compile, its backward compatibility and minimal code changes are particularly advantageous. Graph-level optimization can be applied through a single line of code, enabling performance improvements without altering model architecture.

B. Experiment Setup

We conducted our experiments on a cloud-based virtual machine created through Vast.ai. The instance was configured with an NVIDIA H100 PCIE, representing a modern, transformer optimized GPU that is well suited for evaluating performance optimizations on large, language-based, protein structure prediction models such as ESMFold.

1) Profiling: We profile ESMFold using PyTorch's built-in tool, torch.profiler, rather than relying on CPU wall-clock timing (e.g., time.perf_counter). While perf_counter provides a decent timing estimate that has a low impact on latency, it is not well suited for profiling modern GPU workloads because it cannot be specific enough to discern individual kernels/operations, differentiate between CPU and GPU execution, or expose asynchronous GPU operations get hidden when looking only at the CPU clock. In contrast, torch.profiler provides kernel-level visibility across both CPU and GPU, enabling targeted identification and better understanding of where the bottlenecks lie.

To make the most of the built-in profiler, we enable several optional profiling features. The with_stack option captures Python and C++ call stacks for each operator, allowing use to identify where performance slowdowns are occurring in specific model code lines. The profile_memory option tracks per-operator memory allocation and deallocation behavior, which allows the profiling of peak memory usage. Additionally, we are able to see when stale tensors are being unnecessarily tracked, which leads to expensive and wasteful memory storage that is common to see in long-sequence inference with large pairwise representations. Finally, the record_shapes

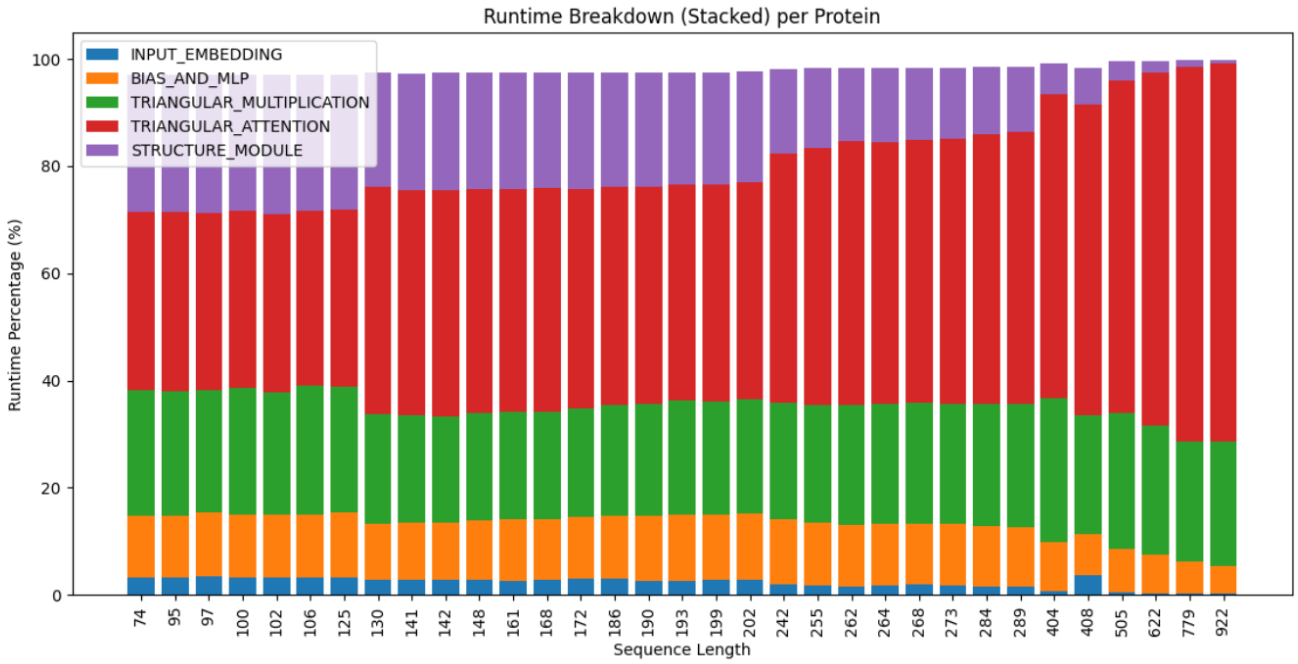

Fig. 2. torch.compile results of total wall clock time on forward pass

option records input tensor dimensions for each operator, providing us with valuable information on input/output tensor shapes, which can be used to identify suboptimal kernel selection or missed opportunities to fuse operations together. [4]

For each experiment we profiled each individual step of the forward pass, then later categorized them into one of five stages: Input Embeddings, Bias and MLP Calculation, Triangular Multiplication, Triangular Attention, and finally, Structural Module. The middle three stages together make up the folding trunk of the model, with the first and last stages representing the pre/post-processing respectively.

The relevant metrics were then logged with Weights and Biases (wandb).

2) Data: In order to fairly evaluate performance on realistic and diverse inputs, we conduct all profiling experiments using the 33 different protein sequences from the CASP14 (Critical Assessment of Structure Prediction, v14) dataset. CASP14 is a widely used benchmark dataset in protein structure prediction due to its inclusion of a wide distribution of structural classes and sequence complexities found in biology.

A key advantage of CASP14 for performance profiling is its wide distribution of sequence lengths, encompassing both short proteins, which we define here as $<$200 residues, and long proteins that can exceed 1000 residues. This diversity is particularly important for analyzing ESMFold, whose computational and memory costs scale quadratically with sequence length due to the construction of pairwise representations and subsequent attention blocks. By profiling across a wide range of sequence lengths, we are able to track how the runtime distribution over model blocks changes, and then identify which of those stages dominate runtime and memory usage when processing long-sequence inputs. [1]

By using CASP14 sequences, we also ensure that the workloads we profile are accurate reflections of real inference workloads on actual biological sequences and not synthetic ones with impossible structures or fixed sequence lengths. We expect the short sequences to be most affected by fixed slowdowns resulting from computing input embeddings, the expensive setup of kernels, and overhead from the compilation of execution graphs. By contrast, long sequences should be controlled by GPU memory availability, the many kernel operations that arise from transformer-based attention kernels, and the computation of structural coordinates in the final stage of prediction. [1] By using real biological sequences, we are able to get a comprehensive characterization of performance bottlenecks on input sequences of varying length while still maintaining realistic inference difficulty.

C. Metrics

The main metrics tracked with each stage were CPU/GPU time, change in memory allocation, and percentage of total forward pass runtime. In addition, to verify that model performance was not affected, three performance metrics were computed including plddt (predicted Local Distance Difference Test), ptm (predicted Template Modeling score), and TM-score (Template Modeling score). plddt is a confidence measure on a per-residue basis that estimates the local accuracy of the predicted structure on a scale from 0 to 100. ptm is a similar metric that focuses on global topology instead of per-residue and is on a scale from 0 to 1. [1] Both of these scores do not use ground-truth labels and are computed using

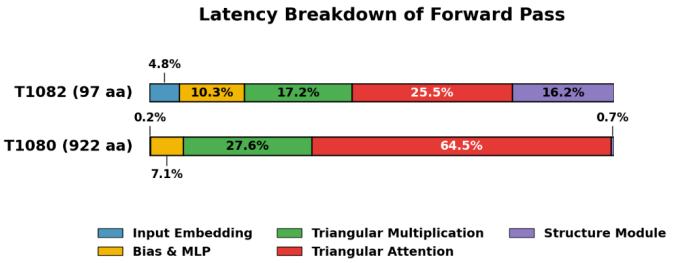

Fig. 3. Comparison of baseline latency broken down by stage. Note the domination of triangular attention and shrinkage of input embedding and structure module.

confidence estimators that are derived from the geometric distribution of the training set. TM-score on the other hand, is computed with a known reference structure and is thus the more objective measure of the three, with the caveat of much a larger computational load as well as requiring the often hard to find and experimentally expensive ground truth structure.

RESULTS

Profiling the baseline model with the CASP14 dataset showed the domination of the triangular attention stage as sequence length increased. Shown on Fig. 2, in short (<200 residue) sequences, triangular attention took up only around 30% of the total runtime, while in longer proteins it jumps up to an average of 47%, with especially long proteins (∼1000 residues) reaching up to 70% runtime (Fig. 3). (Note that the percentages across the five stages do not perfectly sum to 100% due to some intermediary sequences not being tracked.)

TABLE I
CHANGE IN TOTAL WALL CLOCK RUNTIME (tt) WHEN ENABLING TORCH.COMPILE (ROWS WITH $|\Delta tt| > 20\%$ ARE SHOWN IN BOLD)

Target	Len	Baseline tt (s)	Compile tt (s)	Δtt (s)	Δtt (%)
T1046s1	74	3.32	3.23	-0.10	-2.96%
T1031	**95**	**3.30**	**1.91**	**-1.39**	**-42.02%**
T1082	**97**	**2.09**	**3.27**	**+1.19**	**+56.99%**
T1033	**100**	**3.26**	**1.93**	**-1.33**	**-40.69%**
T1035	**102**	**3.29**	**2.03**	**-1.26**	**-38.21%**
T1064	**106**	**1.89**	**3.27**	**+1.38**	**+73.21%**
T1029	125	3.32	3.22	-0.10	-3.02%
T1040	130	3.64	3.58	-0.06	-1.71%
T1046s2	**142**	**2.18**	**3.59**	**+1.41**	**+65.05%**
T1049	141	3.76	3.59	-0.17	-4.44%
T1043	148	3.73	3.61	-0.12	-3.10%
T1039	161	3.61	3.59	-0.02	-0.58%
T1027	**168**	**3.65**	**2.10**	**-1.55**	**-42.46%**
T1026	**172**	**2.20**	**3.62**	**+1.42**	**+64.53%**
T1056	186	3.65	3.70	+0.05	+1.24%
T1054	**190**	**3.71**	**2.41**	**-1.30**	**-35.05%**
T1090	**193**	**3.65**	**2.21**	**-1.44**	**-39.40%**
T1038	199	3.61	3.65	+0.04	+1.09%
T1074	202	3.70	3.60	-0.10	-2.61%
T1041	242	3.71	3.70	-0.01	-0.23%
T1073	255	3.78	3.82	+0.03	+0.90%
T1099	**262**	**2.88**	**4.03**	**+1.14**	**+39.69%**
T1025	268	4.19	4.15	-0.04	-1.05%
T1067	**264**	**2.90**	**4.09**	**+1.19**	**+40.88%**
T1030	**273**	**4.13**	**3.09**	**-1.04**	**-25.27%**
T1032	284	3.60	4.19	+0.59	+16.25%
T1042	**289**	**3.36**	**4.15**	**+0.79**	**+23.50%**
T1037	404	6.64	6.88	+0.24	+3.64%
T1024	408	8.62	7.06	-1.56	-18.09%
T1079	505	11.12	11.14	+0.02	+0.22%
T1036s1	622	18.91	18.96	+0.05	+0.25%
T1050	779	34.89	34.82	-0.07	-0.19%
T1080	922	54.86	54.98	+0.12	+0.22%

D. Comparison between baseline and torch.compile

Between the baseline and compiled models, we see an average of 6.2% speedup in short sequences, but actually a 0.8% slowdown for long sequences (Tab. **??**). However if we look at the per-sequence results, we see that the optimization is not uniform. Some sequences receive a speed up in their total wall clock runtime by 42% (target T1031), but another sequence just two residues longer had a 57% slowdown (target T1082). As a whole, it seems that short-medium length sequences (<500 residues) see the most change in runtime, where the truly long sequences have negligible difference.

By looking at the traces of a specific protein sequence on wandb, we can identify certain steps that had the greatest change between the baseline and torch.compile. For test protein T1082, with a short sequence of only 97 residues and experienced a 57% slowdown, the greatest difference came from the folding trunk iteration steps, where the compilation with torch.compile led the total CPU time to grow from 1630 ms to 2595 ms, a 59% increase in runtime. Other metrics such as GPU/CUDA runtime remained relatively constant, as did the metrics for other steps in the forward pass.

Conversely, for a long protein like T1080, with 922 residues, it experienced a negligible 0.22% slowdown. Looking at its trace, no single step yielded much of a change with the greatest coming from small (<1%) increases in CPU time during steps like folding trunk to structure model MLP projections, ESM encoding, and distogram calculation.

DISCUSSION

E. Baseline model observations

While it was expected that triangular attention would dominate overall run times in longer sequences, what was less obvious was the difference in which stage dominated in CPU vs GPU. As seen in Fig. 4, the stage that dominates CPU time is actually triangular multiplication and not attention. For short sequences, the CPU time breakdown is relatively stagnant as a function of length. However as the proteins grow to over 300 residues, triangular multiplication explodes, eventually comprising of over 90% of the CPU runtime for the largest protein. However, since overall CPU time is about half that of GPU time, the primary bottleneck still lies in the triangular attention. However, this is interesting because it shows that there is also a growing bottleneck at large sequences with CPU or memory access. Looking into

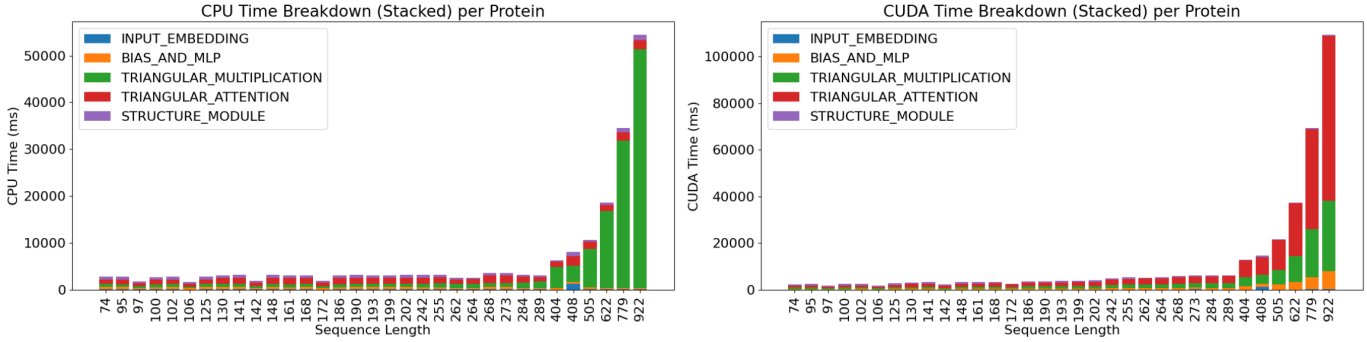

Fig. 4. Comparison of CPU vs CUDA time for forward pass in baseline model

the stack traces with torch.profile reveals the most expensive kernels for triangular multiplication are many repetitive calls to `aten::linear`, `aten::sigmoid`, `aten::mul`, `aten::mul` and `aten::add`. In contrast, for triangular attention there is predominately one single kernel, `aten::flash_attention_forward`, the flash attention kernel. This makes sense as one of the biggest CPU overheads is the expensive kernel startups, and when the CPU is required to make thousands and thousands of small kernel calls, it leads to an accumulation of small slowdowns, whereas if the CPU needs to only run one fused kernel, it passes most of the control time to the GPU.

F. Comparison with torch.compile

This can also explain why torch.compile can lead to such drastic speedups for some of these shorter sequences that are mostly hindered by these fixed CPU overheads. If the compilation can fuse these small kernels together and replace them with a single, large kernel call, it can drastically reduce the amount of startup overhead in the forward pass. We see this in targets T1031, T1033, and T1035, which all get significant speedup, reducing their forward pass time by around 40%.

At the same time, we see that the speedup is not entirely dependent on input sequence length. Some sequences differ by only a few residues, yet they find polarizing changes in their run time (i.e. targets T1033 and T1082). One possible way to explain this is that torch.compile can be detrimental if there are constant graph breaks in the execution. For example, when the code passes through control constructs that depend on tensor values (such as if-else statements), this causes a graph break and forces control to switch back-and-forth between CPU and GPU. If this happens extensively, then torch.compile could perform worse than baseline. In fact, we see this in the folding trunk step that was identified in target T1082 to result in a 59% increase in CPU runtime. In this section of the code, there are calls for `sorted()` and `.items()` on a dictionary, which may not be traceable and thus result in a graph break with torch.compile.

Additionally some of the increase in CPU run time on longer sequences has to be attributed with the profiler itself. With the the model set at a chunking rate of 128 residues, the larger the sequence the more chunk iterations and thus more profiling overhead. If this profiling is fixed, then it will just grow as a function of chunk iterations, and cannot be optimized by torch.compile.

Further Work

Much of this study focused on detailed profiling of the ESMFold model, while making minimal modifications to the underlying architecture. As a result there are a lot of opportunities for numerical optimizations that directly alter the computation of weights and activations, rather than solely improving execution efficiency. One promising direction is the application of quantization-aware techniques such as post-training quantization, which would enable targeted precision reduction in the most memory/compute components without significantly reducing prediction accuracy.

Additionally, we did all of our experiments on a single GPU. The most natural extension would be to use multi-GPU pipelines for larger-scale prediction pipelines that process more than one sequence at a time. Exploring data parallelism while preserving ESMFold's model stability would make it well suited for deployment in large-scale production.

Acknowledgment

Thank you to Professor Kaoutar El Maghraoui and the rest of the teaching staff of COMS-E6998.

References

[1] Z. Lin, H. Akin, R. Rao, B. Hie, Z. Zhu, W. Lu, N. Smetanin, A. dos Santos Costa, M. Fazel-Zarandi, T. Sercu, S. Candido, and A. Rives, "Evolutionary-scale prediction of atomic-level protein structure with a language model," *Science*, vol. 379, no. 6637, pp. 1123–1130, 2023.

[2] S. Marcotte, E. Zhou, A. Patel, I. Lee, W. Zhang, L. Garcia, H. Thompson, M. Nguyen, D. Sokolov, and J. Kim, "LightNobel: Efficient protein folding with long-context attention," *arXiv preprint arXiv:2505.05893*, 2025. [Online]. Available: https://arxiv.org/abs/2505.05893

[3] PyTorch Contributors, "`torch.compile` — PyTorch documentation," 2023. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.compile.html

[4] PyTorch Contributors, "PyTorch profiler — PyTorch documentation," 2023. [Online]. Available: https://pytorch.org/docs/stable/profiler.html